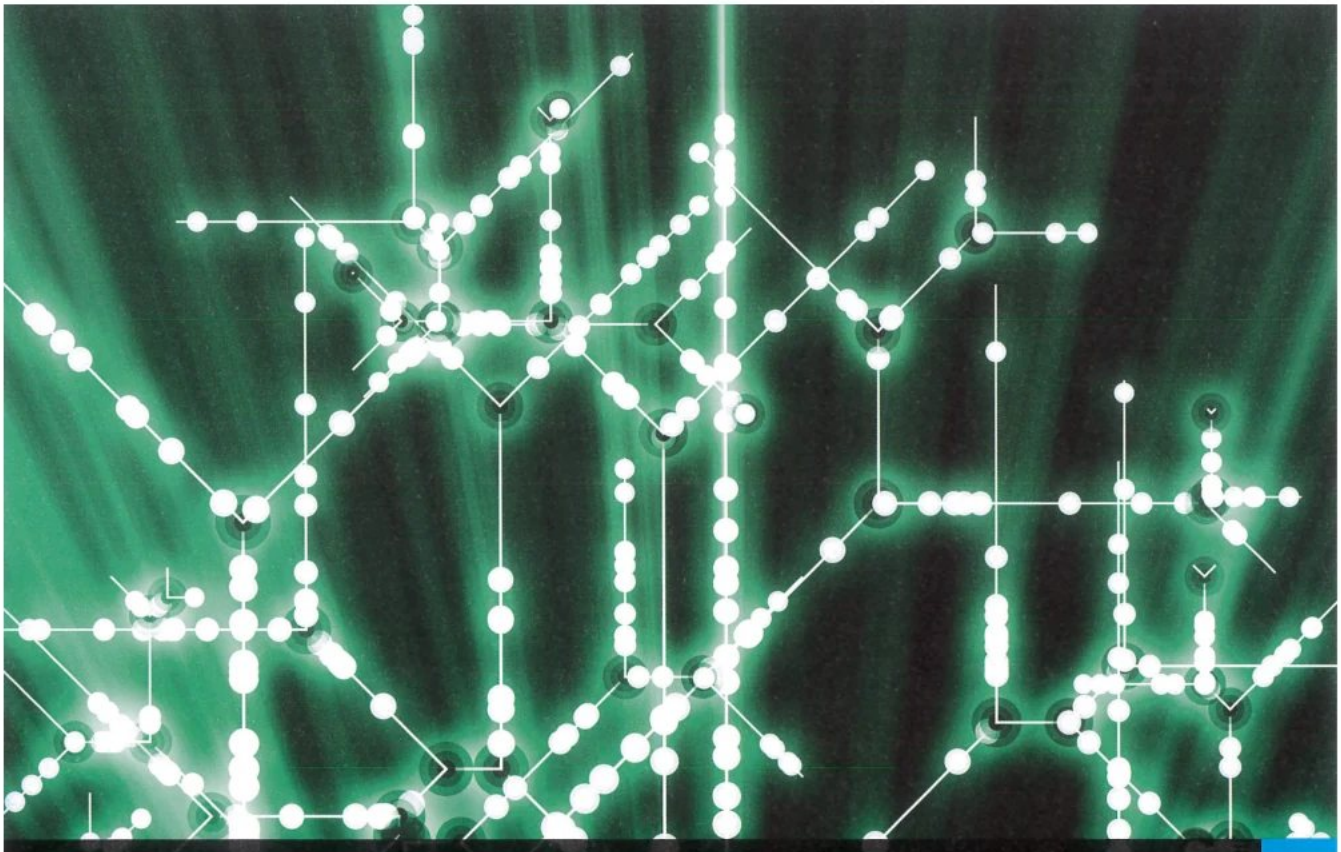


Chapter 16:
Data representation

Learning objectives

By the end of this chapter you should be able to:

- show understanding of why user-defined types are necessary
- define and use non-composite data types
- define and use composite data types
- choose and design an appropriate user-defined data type for a given problem
- show understanding of the methods of file organisation and select an appropriate method of file organisation and file access for a given problem
- show understanding of methods of file access
- show understanding of hashing algorithms
- describe the format of binary floating-point real numbers
- show understanding of the effects of changing the allocation of bits to mantissa and exponent in a floating-point representation
- convert binary floating-point real numbers into denary and vice versa
- normalise floating-point numbers
- show understanding of the consequences of a binary representation only being an approximation to the real number it represents (in certain cases)
- show understanding that binary representations can give rise to rounding errors.

16.01 Data types

Sections 13.01 to 13.05 of Chapter 13 introduced the concept of a variable being associated with a data type. Before a variable can be used in a program, the variable's data type has to be identified. Chapter 13 introduced the most frequently used data types that are available for association with a variable in a program. This chapter introduces some additional data types that might be used.

Built-in data types

Remember, for each built-in data type:

- the programming language defines the range of possible values that can be assigned to a variable when its type has been chosen.
- the programming language defines the operations that are available for manipulating values assigned to the variable.

User-defined data types

The term 'user' is regularly applied to someone who is provided with a 'user interface' by an operating system – the 'user' is the person supplying input to a running program and receiving output from it.

However, when writing a program, a programmer becomes a 'user' of a programming language. The term **user-defined data type** applies to this latter type of user.

A user-defined data type is a data type for which the programmer has included the definition in the program. Once the data type has been defined, variables can be created and associated with the user-defined data type. Note that, although the user-defined data type is not a built-in data type, using the user-defined data type is only possible if a programming language offers support for the construct.



TIP

Make sure that you do not confuse user-defined data types and abstract data types (defined in Section 13.07 of Chapter 13).

Non-composite data types

A **non-composite data type** is one which has a definition which does not involve a reference to another data type. The simple built-in data types, such as integer or real, are examples of built-in non-composite data types. It is also possible for a user-defined data type to be non-composite.

Enumerated data type

An enumerated data type is an example of a user-defined non-composite data type. When a specific **enumerated data type** is defined, every single possible value for it is identified. The following pseudocode shows two examples of enumerated data type definitions:

```
TYPE
TDirections = (North, East, South, West)
TDays = (Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday)
```

Following these definitions, variables can be declared and assigned values, for example:

```
DECLARE Direction1 : TDirections
DECLARE StartDay : TDays
Direction1 ← North
StartDay ← Wednesday
```

It is important to note the following points.

- The values of the enumerated type look like string values but they are not. The values must not be enclosed in quotes.
- The values defined in an enumerated data type are ordinal. This means that enumerated data types have an implied order of values.

The ordering can be put to many uses in a program. For example, a comparison statement can be used with the values of the variables of an enumerated data type:

```
DECLARE Weekend : Boolean
DECLARE Day : TDays
Weekend = TRUE IF Day > Friday
```



KEY TERMS

User-defined data type: where the programmer includes the definition in the program

Non-composite data type: a data type defined without reference to another data type

Enumerated data type: a non-composite user-defined data type for which the definition identifies all possible values

The enumerated data type is one reason why user-defined data types are sometimes needed. There could not be a built-in generic definition of an enumerated data type because the possible values would not be known. The values can only be known when the programmer has identified them in the type definition.

Composite user-defined data types

A composite user-defined data type has a definition with reference to at least one other type. There are two very important examples of composite user-defined data type.

1. The **record data type** (introduced in Chapter 13). Although there could be built-in record data types the expectation is for a record data type to be user-defined. This allows the programmer to create record data types with components that precisely match the data requirements of the particular program. Note that Python is a language that does not support the use of a record data type.
2. The **class**. A class is a data type which is used for an object in object-oriented programming. For a given object-oriented programming language there are likely to be a number of built-in classes. However, if a programmer intends to utilise the benefits of the object-oriented approach, then the programmer will have to create a number of user-defined classes.

Pointer data type

A **pointer variable** is one for which the value is a reference to a memory location. The following is a commentary on some examples of pseudocode involving the use of the pointer data type.

1 An example of the definition of a pointer type which requires only the identification of a data type for which the pointer is to be used.

```
TYPE
TIntegerPointer ← ^Integer
```

2 An example of the declaration of a variable of the pointer data type which does not require the use of the caret (^) symbol.

```
DECLARE MyIntegerPointer : TIntegerPointer
```

3 An example of the declaration of two ordinary variables of type integer and the assignment of a value for one of them.

```
DECLARE Number1, Number2 : INTEGER
Number1 ← 100
```

4 An example of an assignment to a pointer variable of a value which is the address of a different variable.

```
MyIntegerPointer ← @Number1
```

5 An example of an assignment which uses the 'dereferenced' value which has been stored at the address defined by the pointer variable. This assigns the value 200 to Number2.

```
Number2 ← MyIntegerPointer^ * 2
```



KEY TERM

Pointer variable: one for which the value is the address in memory of a different variable

Not all programming languages offer support for the use of a pointer data type. Those languages that do so will have their own version of the symbolism illustrated above with ^ and @.

Because arithmetic can be performed on pointer variables, it is possible to use pointer variables to construct dynamically varying data structures. For some programming languages it is necessary to declare an array with a large upper bound to ensure that the array is unlikely to be fully populated with values. If the language supports the use of a pointer variable, the size of an array can expand while a program is running. The details of how this can be done are beyond the scope of this discussion.

Set data type

A **set** data type allows a program to create sets and to apply the mathematical operations defined in set theory. A set is a mathematical concept with important properties.

- It contains a collection of data values.
- There is no organisation of the data values within the set.
- Duplicate values are not allowed.
- Operations that can be performed on a set include:
  - checking if a value exists in a set
  - adding a new data value
  - removing an existing data value
  - adding one set to another set.

A set variable can be created if a programming language supports the set data type. It is difficult to classify the set data type. Because the set contains multiple components it is tempting to say that the set is a structured data type. However, this contradicts the fact that the set has no structure and therefore no indexing can be associated with the members of the set.

KEY TERM

Set: a collection of data items that lacks any structure; contains no duplicates and has a number of defined operations that can be performed on it

The most useful property of a set is the fact that duplicate values are not allowed. A list or a one-dimensional array might be created but has to be checked to remove duplicate values. A simple way of removing duplicate values would be to convert the structure to a set and then convert the set back to the original structure.

A slightly different example would be if students were allocated to groups for studying a particular subject. For each subject, the students' names would be entered into a data structure defined for that subject. Set data types could then, for example, find out which students were studying both computer science and physics. The students studying both subjects would be found by applying the 'intersection' operation on the two individual sets.

16.02 File organisation

In everyday computer usage, a wide variety of file types is encountered. Examples are graphic files, word-processing files, spreadsheet files and so on. Whatever the file type, the content is stored using a specific binary code that allows the file to be used as intended.

For the specific task of storing data to be used for input to a computer program or for output from a computer program, there are only two defined file types. A file is either a text file or a **binary file**.

A text file, as discussed in Chapter 13 (Section 13.06) contains data stored according to a character code of the type described in Chapter 1 (Section 1.04). It is possible, by using a text editor, to create a text file to be used as input to a program.

A program may create a binary file as output with the intention of subsequently using it for input. A binary file stores data in its internal representation, for example an integer value might be stored in two bytes in two's complement representation.

The organisation of a binary file is based on the concept of a **record**. A file contains records and each record contains fields. Each field consists of a value. For a text file the number of data items per line must be known and the number of characters per item must be known. If these are not known then item separator characters must be used. The file has repeating lines which are defined by an end-of-line character or characters.

For a binary file the number of fields per record must be known. If any of the fields represent a string, the length of the string must be known. For any other field the internal representation will define the number of bytes required to store the field value. There is no need for field separator characters or for an end-of-record character.

KEY TERMS

Binary file: a file designed for storing data to be used by a computer program

Record: a collection of fields containing data values

Discussion Point:

A record is a user-defined data type. It is also a component of a file. Can there be or should there be any relationship between these two concepts?